

Keras Functional Model

➤ What is the Keras Functional API?

Keras offers **two main ways** to build neural networks:

1. **Sequential API**
2. **Functional API**

Let's understand the difference and why the **Functional API** exists.

1. The Sequential Model

The **Sequential model** is like stacking blocks **one on top of another** — layer after layer. It's easy to use but only works for **linear (straight-line)** architectures.

Example:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(32, input_shape=(64,)),
    Dense(10, activation='softmax')
])
```

➤ **Advantages:**

- Very easy to read and write.
- Great for simple models (input → hidden → output).

• **Limitations:**

- Cannot handle **multiple inputs or outputs**.
 - Cannot share layers between models.
 - Cannot create **non-linear architectures** (like networks that branch or merge).
-

2. The Functional API — For Non-linear & Flexible Models

The **Functional API** allows you to design **any type of network**, including:

- Models with **multiple inputs or outputs**
- Models where layers **branch** or **merge**
- Models with **shared layers**
- Models that reuse parts of other models

It treats a neural network as a **graph of layers** rather than a straight line.

Think of it like a **flowchart** data flows from input → through layers → to output, and you can create any shape of connections.

➤ How It Works — Step-by-Step

Here's the key idea:

1. You **define an Input layer**
2. You **apply layers** on top of it (each layer is called as a function)
3. You **connect** layers by **passing one layer's output into another**
4. Finally, you **define a Model** by specifying inputs and outputs.

Example:

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```
# Step 1: Define Input
```

```
inputs = Input(shape=(64,))
```

```
# Step 2: Apply Layers
```

```
x = Dense(32, activation='relu')(inputs)
```

```
outputs = Dense(10, activation='softmax')(x)
```

```
# Step 3: Create Model
```

```
model = Model(inputs=inputs, outputs=outputs)
```

This model works the same as a Sequential one, but now we have full control to create **complex architectures**.

➤ The Problem with Sequential Model

Imagine you want to build:

- A model with **two different inputs**, like **image + text**.
- A model with **multiple outputs**, like **age + gender prediction**.
- A model where one layer's output is sent to **two other layers**.

The **Sequential API cannot handle this**, it assumes one input and one output, in a straight line.

That's why the **Functional API** is needed, it builds **graphs**, not just **chains**.

➤ Simple Example: Functional API in Action

Let's build a small neural network with two hidden layers using the Functional API.

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```
input_layer = Input(shape=(16,))
hidden1 = Dense(8, activation='relu')(input_layer)
hidden2 = Dense(4, activation='relu')(hidden1)
output_layer = Dense(1, activation='sigmoid')(hidden2)

model = Model(inputs=input_layer, outputs=output_layer)
model.summary()
```

Everything is **connected manually**, so we can control how data flows.

➤ Complex Example — Branching Model

Let's say we want a model that splits into two paths and later merges again.

Example:

```
from tensorflow.keras.layers import Input, Dense, concatenate
from tensorflow.keras.models import Model

input_layer = Input(shape=(64,))

# Branch 1
x1 = Dense(32, activation='relu')(input_layer)
x1 = Dense(16, activation='relu')(x1)

# Branch 2
x2 = Dense(32, activation='tanh')(input_layer)
x2 = Dense(16, activation='tanh')(x2)

# Merge both branches
merged = concatenate([x1, x2])
output = Dense(1, activation='sigmoid')(merged)

model = Model(inputs=input_layer, outputs=output)
model.summary()
```

Here you see, Functional API allows you to:

- Create **branches**
 - **Merge** them later
- This is **not possible** with the Sequential API.
-

➤ Multi-Input Model Example

Suppose you want a model that takes **two inputs** —
e.g., an image (pixel data) and some text or numeric data.

```
from tensorflow.keras.layers import Input, Dense, concatenate
from tensorflow.keras.models import Model

# Two separate inputs
image_input = Input(shape=(128,))
text_input = Input(shape=(32,))

# Process both
x1 = Dense(64, activation='relu')(image_input)
x2 = Dense(32, activation='relu')(text_input)

# Combine them
merged = concatenate([x1, x2])
output = Dense(1, activation='sigmoid')(merged)

model = Model(inputs=[image_input, text_input], outputs=output)
model.summary()
```

The Functional API lets you define **multiple input pipelines** and merge them easily.

➤ Multi-Output Model Example

Let's say you want to predict **both price and category** from the same input.

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model

input_layer = Input(shape=(32,))
shared = Dense(64, activation='relu')(input_layer)

# Two separate outputs
price_output = Dense(1, name='price_output')(shared)
category_output = Dense(10, activation='softmax', name='category_output')(shared)

model = Model(inputs=input_layer, outputs=[price_output, category_output])
model.summary()
```

This shows how Functional API easily supports **multiple outputs**.

➤ Why is it Called a “Graph” of Layers?

Because each layer is a **node**,
and each connection is an **edge** in a **directed acyclic graph (DAG)**.
That’s why the Functional API can represent **any kind of structure**,
whereas the Sequential API can only represent a **straight chain**.

➤ When to Use Sequential vs Functional API

Situation	Use Sequential	Use Functional
Simple, linear model	✓	
One input, one output	✓	
Multiple inputs/outputs		✓
Shared layers		✓
Branching/merging		✓
Non-linear architectures		✓

➤ Code Example Summary

Functional API follows **4 key steps**:

1. **Define Inputs** → `Input(shape=(...))`
 2. **Apply Layers** → Each layer is a callable function
 3. **Define Outputs** → Connect final layer(s)
 4. **Create Model** → `Model(inputs, outputs)`
-

➤ In Simple Words

Think of:

- **Sequential Model** = straight line
- **Functional API** = flexible flowchart

The Functional API gives you the **freedom to design any structure** you can imagine —
whether your model **splits, merges, or handles multiple data types**.
